Name: D. Gnana Deepthi

Reg No: 192411123

Marks: /100

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS Marks: 50			Total
Q.No	**Question**	**Bloom's Level**	**Marks**
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	**5**

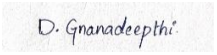
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	**5**
3	Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	**5**
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	**5**
5	Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	**5**
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	**5**
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	**5**
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	**5**
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	**5**
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	**5**

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



ANSWERS

1. STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

we identify the functional dependencies based on what each attribute represents.

1. Functional Dependencies

Assume:

- SID = Student ID
- SName = Student Name
- CID = Course ID
- CName = Course Name
- Instructor = Course Instructor
- Grade = Grade obtained by the student

The functional dependencies are:

1. SID → SName
   A student ID uniquely determines the student name.
2. CID → CName
   A course ID uniquely determines the course name.
3. CID → Instructor
   A course ID uniquely determines its instructor.
4. (SID, CID) → Grade
   A student's grade is determined by the combination of student ID and course ID.

Therefore, the main set of functional dependencies is:

$$\boxed{SID \rightarrow SName}$$

$$\boxed{CID \rightarrow CName,\ Instructor}$$

$$\boxed{SID,\ CID \rightarrow Grade}$$

2. Finding the Candidate Key

We need a set of attributes that can determine all attributes in the relation.

Try (SID, CID):

$$SID, CID \rightarrow SName$$

Because:

$$SID \rightarrow SName$$

Also,

$$SID, CID \rightarrow CName, Instructor$$

Because:

$$CID \rightarrow CName, Instructor$$

And:

$$SID, CID \rightarrow Grade$$

Therefore:

$$\boxed{(SID, CID)^{+} = \{SID, SName, CID, CName, Instructor, Grade\}}$$

So (SID, CID) determines every attribute.

Neither SID nor CID alone can determine all attributes, so the combination is minimal.

2. Conversion of an Unnormalized Relation into 1NF

1. Introduction

First Normal Form (1NF) is the first stage of database normalization. A relation is said to be in **1NF** when:

- Each attribute contains **atomic (single) values**.
- There are **no repeating groups or multivalued attributes**.
- Each row represents a unique record.

2. Unnormalized Relation (UNF)

Consider a **STUDENT** relation:

SID	SName	Courses	Grades
101	Deepthi	DBMS, OS, Python	A, B+, A
102	Priya	DBMS, Java	A+, B
103	Rahul	OS	B+

Here, **Courses** and **Grades** contain multiple values in a single cell.

For example:

SID = 101

- Courses = DBMS, OS, Python
- Grades = A, B+, A

This violates the atomicity requirement of 1NF.

3. Step 1: Identify Repeating Groups

The repeating groups in the relation are:

- **Courses**
- **Grades**

A student can have multiple courses and corresponding grades.

Therefore, these attributes must be separated into individual values.

4. Step 2: Create Separate Rows

Instead of storing multiple courses in one row, create **one row for each student-course combination**.

The relation becomes:

SID	SName	Course	Grade
101	Deepthi	DBMS	A
101	Deepthi	OS	B+
101	Deepthi	Python	A
102	Priya	DBMS	A+
102	Priya	Java	B
103	Rahul	OS	B+

5. Step 3: Identify the Primary Key

A student's **SID alone** cannot uniquely identify a row because one student can take multiple courses.

For example:

- (101, DBMS)
- (101, OS)
- (101, Python)

Therefore, we use the combination:

$$\boxed{(SID, Course)}$$

as the **composite primary key**.

6. Step 4: Verify 1NF

The resulting relation is:

STUDENT_COURSE(SID, SName, Course, Grade)

It satisfies 1NF because:

1. Each cell contains a **single atomic value**.
2. There are **no repeating groups**.
3. Each row represents one **student-course enrollment**.
4. (SID, Course) uniquely identifies each record.

SID	SName	Course	Grade
101	Deepthi	DBMS	A
101	Deepthi	OS	B+
101	Deepthi	Python	A
102	Priya	DBMS	A+
102	Priya	Java	B
103	Rahul	OS	B+

3. Decomposition of R(A, B, C, D, E) into 2NF

Given:

$$R(A, B, C, D, E)$$

Functional Dependencies:

$$A \rightarrow BC$$

$$BC \rightarrow D$$

$$D \rightarrow E$$

Step 1: Find the Candidate Key

Start with A.

From:

$$A \rightarrow B, C$$

we get B and C.

Then:

$$BC \rightarrow D$$

so we get D.

Finally:

$$D \rightarrow E$$

so we get E.

Therefore:

$$A^{+} = \{A, B, C, D, E\}$$

Hence, A is the candidate key.

$$\boxed{\text{Candidate Key} = A}$$

Step 2: Check for Partial Dependencies

A relation is in 2NF if it is in 1NF and there are no partial dependencies of non-key attributes on part of a composite candidate key.

Here, the candidate key is A, which consists of only one attribute.

Therefore, a partial dependency cannot exist, because there is no "part" of the key A.

For example:

$$A \rightarrow B, C$$

is a dependency on the whole candidate key, not a partial dependency.

Step 3: Is Decomposition Required?

No decomposition is required to achieve 2NF.

The original relation:

$$\boxed{R(A, B, C, D, E)}$$

is already in 2NF, assuming it is in 1NF.

However, notice:

$$BC \rightarrow D$$

And

$$D \rightarrow E$$

are transitive dependencies, which are relevant when converting to 3NF, not 2NF.

4. 3NF Decomposition of EMPLOYEE Relation

Given Relation

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Step 1: Identify Functional Dependencies

The functional dependencies are:

1. EmpID → EmpName, DeptID
2. DeptID → DeptName, ManagerID

Since:

EmpID → DeptID and DeptID → DeptName, ManagerID

we can derive:

EmpID → DeptName, ManagerID

This represents a **transitive dependency**:

EmpID → DeptID → DeptName, ManagerID

Step 2: Identify the Candidate Key

EmpID uniquely identifies an employee.

Therefore:

Candidate Key = EmpID

Step 3: Identify the Transitive Dependency

The dependency:

DeptID → DeptName, ManagerID

is a transitive dependency because **DeptID is a non-key attribute** and it determines other non-key attributes.

Thus, the original relation is not in **3NF**.

Step 4: Decompose the Relation

Separate the employee information from the department information.

Relation 1: EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

Functional dependency:

EmpID → EmpName, DeptID

Primary Key: **EmpID**

Relation 2: DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

Functional dependency:

DeptID → DeptName, ManagerID

Primary Key: **DeptID**

Step 5: Final 3NF Relations

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

The **DeptID** in EMPLOYEE acts as a **foreign key** referencing DEPARTMENT(DeptID).

Conclusion

The original EMPLOYEE relation contains the transitive dependency:

EmpID → DeptID → DeptName, ManagerID

To eliminate this dependency, the relation is decomposed into **EMPLOYEE** and **DEPARTMENT**. Both resulting relations satisfy **3NF**, because every non-key attribute depends directly on the key of its respective relation.

5. BCNF Verification and Decomposition

Given Relation

R(A, B, C, D)

Functional Dependencies:

1. **AB → C**
2. **C → D**
3. **D → A**

Step 1: Find Candidate Keys

Check AB

From:

AB → C

we get C.

From:

C → D

we get D.

Therefore:

(AB)+ = {A, B, C, D}

So, **AB is a candidate key**.

Check BC

From:

C → D

we get D.

From:

D → A

we get A.

Therefore:

(BC)+ = {A, B, C, D}

So, **BC is a candidate key**.

Check BD

From:

D → A

we get A.

Now we have A and B, so:

AB → C

gives C.

Therefore:

(BD)+ = {A, B, C, D}

So, **BD is also a candidate key**.

Thus, the candidate keys are:

AB, BC, BD

Step 2: Check BCNF

A relation is in **BCNF** if, for every non-trivial functional dependency:

X → Y

X must be a **superkey**.

FD 1: AB → C

AB is a candidate key.

Therefore, **AB → C satisfies BCNF**.

FD 2: C → D

C is **not a superkey**, because C alone cannot determine A and B.

Therefore, **C → D violates BCNF**.

FD 3: D → A

D is **not a superkey**, because D alone cannot determine B and C.

Therefore, **D → A also violates BCNF**.

Hence:

R is NOT in BCNF.

Step 3: Decompose Using C → D

For the violating dependency:

C → D

create:

R1(C, D)

The remaining attributes form:

R2(A, B, C)

So the decomposition is:

R1(C, D)
R2(A, B, C)

Step 4: Verify BCNF of R1

For **R1(C, D)**:

C → D

C determines all attributes of R1.

Therefore, C is a candidate key of R1.

Hence, **R1 is in BCNF**.

Step 5: Verify BCNF of R2

For **R2(A, B, C)**:

The relevant dependency is:

AB → C

AB determines all attributes of R2.

Therefore, AB is a candidate key of R2.

Hence, **R2 is in BCNF**.

Final BCNF Decomposition

R1(C, D)

- Candidate Key: **C**
- FD: **C → D**
- BCNF: **Yes**

R2(A, B, C)

- Candidate Key: **AB**
- FD: **AB → C**
- BCNF: **Yes**

Final Answer

The original relation **R(A,B,C,D) is not in BCNF** because **C → D** and **D → A** have determinants that are not superkeys.

After decomposition:

R(A,B,C,D)

↓ Decompose using C → D

R1(C,D) + **R2(A,B,C)**

Both resulting relations are in **BCNF**.

6. **Experimental Verification of Lossless-Join Decomposition Using Chase Algorithm**

Aim

To experimentally verify whether a given decomposition of a relation is lossless-join using the tabular (Chase) algorithm.

Given Relation

Consider the relation:

R(A, B, C)

with the functional dependency:

A → B

Decompose R into:

R1(A, B)

R2(A, C)

We need to determine whether this decomposition is **lossless**.

Step 1: Create the Chase Table

Create a table with one row for each decomposed relation and one column for each attribute of R.

For attributes belonging to a relation, use the corresponding common symbol. For other attributes, use a distinct symbol.

Relation	**A**	**B**	**C**
R1(A,B)	a	b	c_1
R2(A,C)	a	b_2	c

Here:

- a represents the common value of A.
- b represents the value of B in R1.
- c represents the value of C in R2.
- c_1 and b_2 are distinct symbols.

Step 2: Apply the Functional Dependency

Given:

A → B

Both rows have the same value **a** for attribute A.

Therefore, according to **A → B**, their B values must become equal.

So, replace **b_2** with **b**.

The table becomes:

Relation	**A**	**B**	**C**
R1(A,B)	a	b	c_1
R2(A,C)	a	b	c

Step 3: Check the Chase Result

After applying the functional dependency, the second row becomes:

(a, b, c)

This row contains the distinguished symbols **a, b, c** for all attributes of R.

Therefore, the chase succeeds.

Result

Since a row containing all distinguished symbols is obtained, the decomposition is **lossless-join**.

$$
\boxed{R(A,B,C) \rightarrow R_1(A,B), R_2(A,C)\text{ is lossless}}
$$

Step 4: Experimental Verification

The decomposition can also be verified conceptually by joining the decomposed relations.

Suppose:

R1

A	**B**
1	X

R2

A	**C**
1	Y

Joining R1 and R2 on A gives:

A	**B**	**C**
1	X	Y

The original tuple is reconstructed without generating an unwanted tuple.

Therefore, the decomposition is experimentally verified as **lossless-join**.

7. Identification of Multi-Valued Dependencies and 4NF Decomposition

Given Relation

TEACHER(TID, Subject, Hobby)

Where:

- **TID** = Teacher ID
- **Subject** = Subject taught by the teacher
- **Hobby** = Hobby of the teacher

Step 1: Identify the Multi-Valued Dependencies

Assume that a teacher can teach multiple subjects and can have multiple hobbies, and these two sets of values are independent of each other.

Therefore, the following multi-valued dependencies exist:

TID →→ Subject

TID →→ Hobby

This means that for a particular teacher, the subjects they teach are independent of their hobbies.

Example

Suppose teacher T01 teaches:

- DBMS
- Operating Systems

and has hobbies:

- Music
- Photography

The original relation may contain:

TID	Subject	Hobby
T01	DBMS	Music
T01	DBMS	Photography
T01	Operating Systems	Music

TID	Subject	Hobby
T01	Operating Systems	Photography

The combinations are generated because **Subject** and **Hobby** are independent multi-valued attributes.

Step 2: Check for 4NF Violation

A relation is in Fourth Normal Form (4NF) if, for every non-trivial multivalued dependency:

$X \rightarrow\rightarrow Y$

X must be a superkey.

Here:

$TID \rightarrow\rightarrow Subject$

and

$TID \rightarrow\rightarrow Hobby$

But TID alone is not a superkey of TEACHER because it does not uniquely identify a single subject-hobby combination.

Therefore, the relation violates 4NF.

Step 3: Decompose the Relation

To remove the multi-valued dependencies, decompose TEACHER into two relations.

Relation 1: TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

This represents the subjects taught by each teacher.

MVD:

$TID \rightarrow\rightarrow Subject$

Primary Key:

(TID, Subject)

Relation 2: TEACHER_HOBBY

TEACHER_HOBBY(TID, Hobby)

This represents the hobbies of each teacher.

MVD:

TID →→ Hobby

Primary Key:

(TID, Hobby)

Step 4: Verify 4NF

TEACHER_SUBJECT(TID, Subject)

The relation contains only the teacher and subject information. There is no independent multi-valued attribute remaining.

Therefore, it is in **4NF**.

TEACHER_HOBBY(TID, Hobby)

The relation contains only the teacher and hobby information. There is no independent multi-valued attribute remaining.

Therefore, it is in **4NF**.

8. Decomposition of a Relation into 5NF (Project-Join Normal Form)

Aim

To demonstrate how join dependencies can cause redundancy and decompose a given relation into Fifth Normal Form (5NF), also called Project-Join Normal Form (PJ/NF).

Given Relation

Consider the relation:

SUPPLIER_PART_PROJECT(Supplier, Part, Project)

Where:

- Supplier = Supplier who provides the part
- Part = Part supplied
- Project = Project for which the part is supplied

Assume that the relationship among Supplier, Part, and Project can be completely represented by three pairwise relationships:

- Supplier supplies Part
- Supplier works for Project
- Part is used in Project

This creates a join dependency.

Step 1: Identify the Join Dependency

The relation has the join dependency:

(Supplier, Part), (Supplier, Project), (Part, Project)

This means that the original three-way relationship can be reconstructed by joining the following projections:

SP(Supplier, Part)

SJ(Supplier, Project)

PJ(Part, Project)

Therefore:

[
R = SP \bowtie SJ \bowtie PJ
]

Step 2: Demonstrate Redundancy

Suppose the original relation contains:

Supplier	**Part**	**Project**
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

The same supplier-part relationship and supplier-project relationship are repeated across multiple tuples.

For example:

- S1 supplies P1 and P2.
- S1 works on J1 and J2.

The combinations create repeated information.

This is a form of join redundancy.

Step 3: Decompose the Relation

To remove the redundancy, decompose the relation into three smaller relations.

Relation 1: SUPPLIER_PART

SUPPLIER_PART(Supplier, Part)

Supplier	Part
S1	P1
S1	P2

Relation 2: SUPPLIER_PROJECT

SUPPLIER_PROJECT(Supplier, Project)

Supplier	Project
S1	J1
S1	J2

Relation 3: PART_PROJECT

PART_PROJECT(Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1

Part	Project
P2	J2

Step 4: Reconstruct the Original Relation

The original relation can be reconstructed by joining the three decomposed relations:

[
R =
SUPPLIER_PART
\bowtie
SUPPLIER_PROJECT
\bowtie
PART_PROJECT
]

The decomposition preserves the join dependency and does not lose information.

Step 5: Verify 5NF

A relation is in 5NF (PJ/NF) if every non-trivial join dependency is implied by the candidate keys of the relation.

In the original relation, the three-way relationship can be represented by the three pairwise relations. Therefore, the join dependency is separated into independent relations.

The decomposed relations:

- SUPPLIER_PART
- SUPPLIER_PROJECT
- PART_PROJECT

do not contain the problematic non-trivial join dependency.

Hence, the decomposition satisfies 5NF under the given business assumption.

9. Identification of Anomalies and Normalization to 3NF

Aim

To identify insertion, deletion, and update anomalies in a poorly designed college database schema and normalize the relation up to Third Normal Form (3NF).

1. Poorly Designed Relation

Consider the following college relation:

COLLEGE(StudentID, StudentName, DeptID, DeptName, CourseID, CourseName, Instructor, Grade)

Sample data:

StudentID	StudentName	DeptID	DeptName	CourseID	CourseName	Instructor	Grade
S101	Deepthi	D01	CSE	C01	DBMS	Kumar	A
S101	Deepthi	D01	CSE	C02	OS	Ravi	B+
S102	Priya	D01	CSE	C01	DBMS	Kumar	A+
S103	Rahul	D02	ECE	C03	Networks	Arun	B

Functional Dependencies

The important functional dependencies are:

StudentID → StudentName, DeptID

DeptID → DeptName

CourseID → CourseName, Instructor

StudentID, CourseID → Grade

The candidate key is:

(StudentID, CourseID)

2. Identify the Anomalies

A. Insertion Anomaly

Suppose a new course C04 – Artificial Intelligence is introduced, but no student has enrolled in it yet.

In the poorly designed relation, we cannot store the course information without providing a StudentID and Grade.

Therefore, we are forced to insert incomplete or NULL values.

Problem: We cannot independently insert information about a new course or department.

B. Deletion Anomaly

Suppose student **S103** is the only student enrolled in course **C03 – Networks**.

If S103's record is deleted, the information about:

- C03
- Networks
- Arun (Instructor)

is also lost.

Similarly, if the only student in a department is deleted, the department information may also disappear.

Problem: Deleting one student can unintentionally delete course or department information.

C. Update Anomaly

Suppose the instructor for **C01 – DBMS** changes from **Kumar** to **Ramesh**.

Since C01 appears in multiple student records, every occurrence must be updated.

If one record is not updated, the database will contain inconsistent information.

For example:

StudentID	CourseID	Instructor
S101	C01	Ramesh
S102	C01	Kumar

Problem: The same information is stored repeatedly, causing update inconsistency.

3. First Normal Form (1NF)

The relation is already in **1NF** because each attribute contains atomic values and there are no repeating groups.

However, it contains partial dependencies because the composite key is:

(StudentID, CourseID)

For example:

StudentID → StudentName, DeptID

and

CourseID → CourseName, Instructor

These attributes depend only on part of the composite key.

Therefore, the relation is not in **2NF**.

4. Convert to 2NF

Remove the partial dependencies by decomposing the relation.

STUDENT

STUDENT(StudentID, StudentName, DeptID)

Primary Key: **StudentID**

COURSE

COURSE(CourseID, CourseName, Instructor)

Primary Key: **CourseID**

ENROLLMENT

ENROLLMENT(StudentID, CourseID, Grade)

Primary Key: **(StudentID, CourseID)**

Foreign Keys:

- StudentID → STUDENT
- CourseID → COURSE

5. Convert to 3NF

Now examine the STUDENT relation:

STUDENT(StudentID, StudentName, DeptID)

We have:

StudentID → DeptID

and

DeptID → DeptName

Therefore:

StudentID → DeptID → DeptName

This is a **transitive dependency**.

To remove it, create a separate DEPARTMENT relation.

DEPARTMENT

DEPARTMENT(DeptID, DeptName)

Primary Key: **DeptID**

STUDENT

STUDENT(StudentID, StudentName, DeptID)

Primary Key: **StudentID**

Foreign Key:

DeptID → DEPARTMENT(DeptID)

6. Final 3NF Schema

The final normalized database consists of four relations:

1. STUDENT

STUDENT(StudentID, StudentName, DeptID)

- Primary Key: StudentID
- Foreign Key: DeptID

2. DEPARTMENT

DEPARTMENT(DeptID, DeptName)

- Primary Key: DeptID

3. COURSE

COURSE(CourseID, CourseName, Instructor)

- Primary Key: CourseID

4. ENROLLMENT

ENROLLMENT(StudentID, CourseID, Grade)

- Primary Key: (StudentID, CourseID)
- Foreign Keys: StudentID, CourseID

10. Attribute Closure and Candidate Keys

Given Question

Compute the closure of attribute sets for the given functional dependencies (FDs) and determine all candidate keys for the relation.

Explanation

The **attribute closure** of an attribute set X, denoted by $\mathbf{X}^+$, is the set of all attributes that can be functionally determined from X using the given functional dependencies.

To compute X^+:

1. Start with $X^+ = X$.
2. Examine each functional dependency.
3. If the left-hand side of an FD is contained in X^+, add its right-hand-side attributes to X^+.
4. Repeat until no new attributes can be added.

An attribute set is a **candidate key** if:

- Its closure contains all attributes of the relation.
- No proper subset of it can determine all attributes.

Example

Consider:

R(A, B, C, D)

Functional dependencies:

A → B

B → C

C → D

Compute $\mathbf{A}^+$:

Initially:

$\mathbf{A}^+ = \{A\}$

Using **A → B**:

$\mathbf{A}^+ = \{A, B\}$

Using **B → C**:

$\mathbf{A}^+ = \{A, B, C\}$

Using **C → D**:

A^+ = {A, B, C, D}

Since A^+ contains all attributes of R:

A is a superkey.

Since A has no proper subset, **A is a candidate key**.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and	Good analysis with reasonable	Basic analysis with limited insights	Poor or incorrect analysis of results

		validation of results	interpretation		
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation